

Universidade de Brasília
Faculdade de Tecnologia

Cálculo Numérico Aplicado

Programa para Casa #2 (PPC2)

Prof. Dr. Rafael Gabler Gontijo

Mateus Leal Silva

Matrícula: 221028134

Brasília, 14 de abril de 2026

Sumário

1	Introdução	2
2	Fundamentação do Método de Bairstow	2
3	Implementação Computacional	3
4	Validação com Polinômio de Sétima Ordem	4
5	Análise de Convergência	4
6	Aplicação ao Polinômio Característico da APC2	5
7	Interpretação Física dos Autovalores	5
8	Fractal de Bairstow	6
9	Código-Fonte Python	7
10	Conclusão	12

1 Introdução

O presente relatório apresenta a implementação computacional do método de Bairstow para determinação de raízes reais e complexas de polinômios de grau arbitrário. O método é aplicado em três frentes principais: validação com um polinômio de sétima ordem com raízes conhecidas, análise da convergência para diferentes valores iniciais dos parâmetros r_0 e s_0 , e solução do polinômio característico obtido na APC2 para um sistema massa-mola-amortecedor com dois graus de liberdade.

Na atividade analítica anterior, o sistema dinâmico levou ao seguinte polinômio característico:

$$P(\lambda) = 2\lambda^4 + 5\lambda^3 + 12\lambda^2 + 8\lambda + 8. \quad (1)$$

O objetivo computacional do PPC2 é determinar as raízes desse polinômio por meio do método de Bairstow e interpretar os autovalores resultantes em termos de estabilidade, amortecimento e frequência de oscilação.

2 Fundamentação do Método de Bairstow

Considere um polinômio real de grau n escrito em ordem crescente de potência:

$$f_n(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n. \quad (2)$$

No método de Bairstow, procura-se dividir $f_n(x)$ por um fator quadrático real da forma:

$$D(x) = x^2 - rx - s, \quad (3)$$

onde r e s são parâmetros ajustados iterativamente. A vantagem dessa escolha é que o método consegue obter raízes complexas conjugadas sem utilizar aritmética complexa durante o processo iterativo principal.

A divisão sintética quadrática é escrita como:

$$f_n(x) = (b_2 + b_3x + \cdots + b_nx^{n-2})(x^2 - rx - s) + [b_0 + b_1(x - r)]. \quad (4)$$

Assim, para que $D(x)$ seja de fato um fator do polinômio, o resto deve ser nulo:

$$b_0 = 0 \quad \text{e} \quad b_1 = 0. \quad (5)$$

Os coeficientes b_i são calculados pelas recorrências:

$$b_n = a_n, \quad (6)$$

$$b_{n-1} = a_{n-1} + rb_n, \quad (7)$$

$$b_i = a_i + rb_{i+1} + sb_{i+2}, \quad i = n-2, n-3, \dots, 0. \quad (8)$$

Para ajustar r e s , aplica-se uma linearização de Newton-Raphson no sistema não linear formado por $b_0(r, s)$ e $b_1(r, s)$. O sistema linear para as correções Δr e Δs é:

$$\begin{bmatrix} \frac{\partial b_1}{\partial r} & \frac{\partial b_1}{\partial s} \\ \frac{\partial b_0}{\partial r} & \frac{\partial b_0}{\partial s} \end{bmatrix} \begin{bmatrix} \Delta r \\ \Delta s \end{bmatrix} = \begin{bmatrix} -b_1 \\ -b_0 \end{bmatrix}. \quad (9)$$

As derivadas parciais são obtidas por um segundo conjunto de recorrências, definido pelos coeficientes c_i :

$$c_n = b_n, \quad (10)$$

$$c_{n-1} = b_{n-1} + rc_n, \quad (11)$$

$$c_i = b_i + rc_{i+1} + sc_{i+2}, \quad i = n-2, n-3, \dots, 1. \quad (12)$$

com as relações:

$$c_1 = \frac{\partial b_0}{\partial r}, \quad c_2 = \frac{\partial b_0}{\partial s} = \frac{\partial b_1}{\partial r}, \quad c_3 = \frac{\partial b_1}{\partial s}. \quad (13)$$

Após resolver o sistema linear, os parâmetros são atualizados por:

$$r_{k+1} = r_k + \Delta r, \quad s_{k+1} = s_k + \Delta s. \quad (14)$$

Quando b_0 e b_1 se tornam suficientemente pequenos, as duas raízes associadas ao divisor quadrático são obtidas por:

$$x = \frac{r \pm \sqrt{r^2 + 4s}}{2}. \quad (15)$$

Em seguida, o polinômio é deflacionado, isto é, substituído pelo quociente $b_2 + b_3x + \dots + b_nx^{n-2}$, e o processo é repetido até que todas as raízes sejam determinadas.

3 Implementação Computacional

A implementação foi desenvolvida em Python utilizando apenas estruturas básicas da linguagem, vetores numéricos e rotinas gráficas. A rotina principal trabalha com os coeficientes em ordem crescente de potência, isto é:

$$[a_0, a_1, a_2, \dots, a_n]. \quad (16)$$

Para o polinômio da APC2, a entrada utilizada foi:

$$[8, 8, 12, 5, 2]. \quad (17)$$

Os critérios numéricos adotados foram:

- tolerância para atualização dos parâmetros: 10^{-10} ;
- tolerância para o resto da divisão: 10^{-8} ;
- número máximo de iterações por fator quadrático: 100.

O programa foi organizado em funções para: divisão sintética quadrática, cálculo dos coeficientes auxiliares c_i , iteração de Bairstow para um fator quadrático, deflação polinomial, solução do polinômio completo, análise de convergência e geração do fractal de Bairstow.

4 Validação com Polinômio de Sétima Ordem

Para validar a implementação, foi construído um polinômio de grau sete com raízes previamente conhecidas:

$$x_r = \{-3, -1, 2, 4, 5, 1 + 2i, 1 - 2i\}. \quad (18)$$

A multiplicação dos fatores lineares e quadrático correspondentes gera:

$$p_v(x) = x^7 - 9x^6 + 16x^5 + 50x^4 - 219x^3 + 367x^2 + 10x - 600. \quad (19)$$

Portanto, no formato computacional crescente, o vetor de coeficientes é:

$$[-600, 10, 367, -219, 50, 16, -9, 1]. \quad (20)$$

A Tabela 1 mostra as raízes recuperadas pelo programa.

Tabela 1: Validação do método com polinômio de sétima ordem.

Raiz esperada	Raiz obtida	Comentário
2	2.0000000000	real
-1	-1.0000000000	real
5	5.0000000000	real
-3	-3.0000000000	real
$1 + 2i$	$1.0000000000 + 2.0000000000i$	complexa
$1 - 2i$	$1.0000000000 - 2.0000000000i$	complexa conjugada
4	4.0000000000	real

O maior erro absoluto observado, considerando a associação entre as raízes esperadas e as raízes calculadas, foi da ordem de 10^{-13} . Isso indica que a implementação recuperou adequadamente tanto raízes reais quanto raízes complexas conjugadas.

5 Análise de Convergência

A convergência do método foi avaliada para diferentes escolhas iniciais de r_0 e s_0 aplicadas ao polinômio característico da APC2. A Tabela 2 apresenta os resultados obtidos para o primeiro fator quadrático encontrado.

Tabela 2: Influência dos chutes iniciais na convergência do método de Bairstow.

r_0	s_0	Iterações	r final	s final
0	-1	5	-0.50000000	-1.00000000
-1	-1	5	-0.50000000	-1.00000000
-2	-4	1	-2.00000000	-4.00000000
1	1	8	-0.50000000	-1.00000000
-2	1	7	-0.50000000	-1.00000000
2	-1	8	-0.50000000	-1.00000000
3	3	10	-0.50000000	-1.00000000
-3	-3	8	-0.50000000	-1.00000000

Observa-se que o método converge rapidamente quando o par inicial está próximo de um fator quadrático real do polinômio. Por exemplo, o chute $r_0 = -2$ e $s_0 = -4$ já coincide com um dos fatores exatos, resultando em convergência imediata. Chutes mais afastados também convergiram, mas exigiram maior número de iterações.

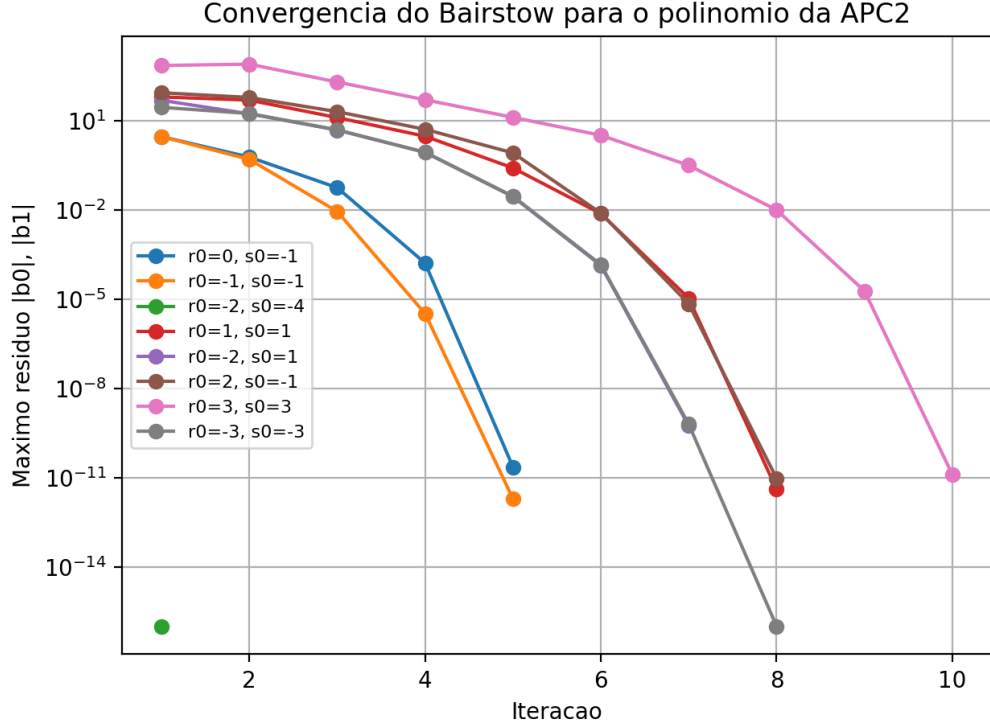


Figura 1: Evolução do resíduo máximo $\max(|b_0|, |b_1|)$ para diferentes chutes iniciais.

6 Aplicação ao Polinômio Característico da APC2

O polinômio obtido na APC2 foi:

$$P(\lambda) = 2\lambda^4 + 5\lambda^3 + 12\lambda^2 + 8\lambda + 8. \quad (21)$$

A aplicação do método de Bairstow identificou os seguintes fatores quadráticos:

$$P(\lambda) = 2(\lambda^2 + 0.5\lambda + 1)(\lambda^2 + 2\lambda + 4). \quad (22)$$

Consequentemente, os autovalores calculados foram:

$$\lambda_{1,2} = -0.2500000000 \pm 0.9682458366i, \quad (23)$$

$$\lambda_{3,4} = -1.0000000000 \pm 1.7320508076i. \quad (24)$$

A substituição dessas raízes no polinômio original fornece resíduo máximo da ordem de 10^{-11} , confirmando numericamente a consistência da solução obtida.

7 Interpretação Física dos Autovalores

Para sistemas dinâmicos lineares amortecidos, um par complexo de autovalores pode ser escrito como:

$$\lambda = \sigma \pm i\omega_d, \quad (25)$$

onde σ é a taxa de decaimento exponencial e ω_d é a frequência amortecida de oscilação.

Para o primeiro par:

$$\lambda_{1,2} = -0.25 \pm 0.9682458366i, \quad (26)$$

tem-se:

$$\sigma_1 = -0.25, \quad \omega_{d,1} = 0.9682458366 \text{ rad/s}. \quad (27)$$

A frequência natural associada é:

$$\omega_{n,1} = \sqrt{\sigma_1^2 + \omega_{d,1}^2} = 1.0 \text{ rad/s}, \quad (28)$$

e o fator de amortecimento modal é:

$$\zeta_1 = \frac{-\sigma_1}{\omega_{n,1}} = 0.25. \quad (29)$$

Para o segundo par:

$$\lambda_{3,4} = -1 \pm 1.7320508076i, \quad (30)$$

tem-se:

$$\sigma_2 = -1, \quad \omega_{d,2} = 1.7320508076 \text{ rad/s}. \quad (31)$$

A frequência natural é:

$$\omega_{n,2} = \sqrt{\sigma_2^2 + \omega_{d,2}^2} = 2.0 \text{ rad/s}, \quad (32)$$

e o fator de amortecimento modal é:

$$\zeta_2 = \frac{-\sigma_2}{\omega_{n,2}} = 0.50. \quad (33)$$

Como todos os autovalores possuem parte real negativa, o sistema é assintoticamente estável. Fisicamente, isso significa que uma perturbação inicial produz oscilações amortecidas, cuja amplitude decai com o tempo. O primeiro modo tem decaimento mais lento, pois $|\sigma_1| = 0.25$, enquanto o segundo modo dissipa energia mais rapidamente, pois $|\sigma_2| = 1$.

8 Fractal de Bairstow

Para avaliar a dependência do método em relação às condições iniciais, foi realizada uma varredura no plano (r_0, s_0) , registrando o número de iterações necessário para a convergência. O mapa foi gerado no intervalo:

$$r_0 \in [-5, 5], \quad s_0 \in [-5, 5]. \quad (34)$$

Cada ponto do plano representa uma execução independente do método de Bairstow. A escala do gráfico indica o número de iterações até a convergência; regiões com maior número de iterações representam maior sensibilidade ao chute inicial.

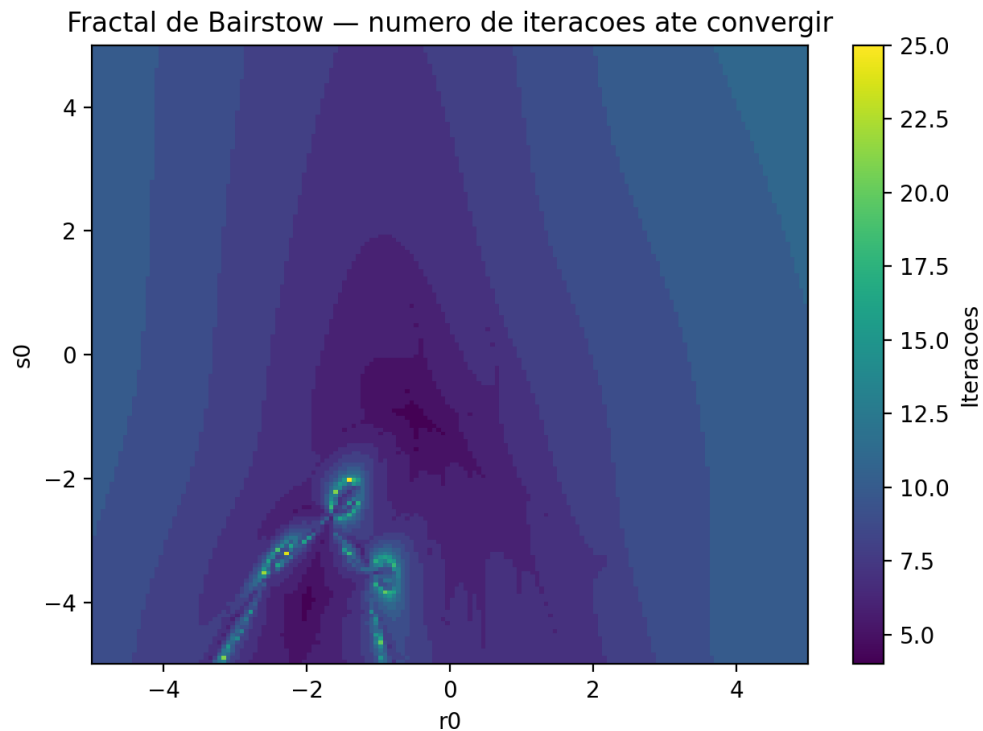


Figura 2: Fractal de Bairstow associado ao polinômio característico da APC2.

O mapa evidencia que o método possui bacias de convergência distintas. Regiões próximas dos fatores quadráticos exatos apresentam convergência mais rápida, enquanto regiões mais afastadas exigem mais iterações ou podem apresentar comportamento mais sensível. Esse resultado está de acordo com a natureza iterativa do método, que utiliza uma correção do tipo Newton–Raphson para ajustar simultaneamente os parâmetros r e s .

9 Código-Fonte Python

A implementação completa utilizada para gerar os resultados e figuras deste relatório é apresentada abaixo.

```

1  """
2  PPC2 - Metodo de Bairstow
3  Calculo Numerico Aplicado
4  Autor: Mateus Leal Silva
5
6  Convencao adotada:
7      p(x) = a0 + a1*x + a2*x^2 + ... + an*x^n
8  ou seja, os coeficientes sao armazenados em ordem crescente de potencia.
9
10 O divisor quadratico do metodo de Bairstow e:
11     D(x) = x^2 - r*x - s
12
13 As raizes associadas ao divisor sao:
14     x = (r +/- sqrt(r^2 + 4s))/2
15 """
16

```



```

17 import cmath
18 import numpy as np
19 import matplotlib.pyplot as plt
20
21
22 def divisao_sintetica_quadratica(a, r, s):
23     """Calcula os coeficientes b_i da divisao sintetica por x^2 - r x -
24     s."""
25     n = len(a) - 1
26     b = np.zeros(n + 1, dtype=float)
27
28     b[n] = a[n]
29     b[n - 1] = a[n - 1] + r * b[n]
30
31     for i in range(n - 2, -1, -1):
32         b[i] = a[i] + r * b[i + 1] + s * b[i + 2]
33
34     return b
35
36 def coeficientes_c(b, r, s):
37     """Calcula os coeficientes c_i usados na linearizacao de Newton-
38     Raphson."""
39     n = len(b) - 1
40     c = np.zeros(n + 1, dtype=float)
41
42     c[n] = b[n]
43     c[n - 1] = b[n - 1] + r * c[n]
44
45     for i in range(n - 2, -1, -1):
46         c[i] = b[i] + r * c[i + 1] + s * c[i + 2]
47
48     return c
49
50 def bairstow_par(a, r0, s0, tol=1e-10, max_iter=100):
51     """
52     Aplica o metodo de Bairstow ate encontrar um fator quadratico.
53
54     Retorna:
55         r, s, b, historico, convergiu
56     """
57     r = float(r0)
58     s = float(s0)
59     historico = []
60
61     for it in range(1, max_iter + 1):
62         b = divisao_sintetica_quadratica(a, r, s)
63         c = coeficientes_c(b, r, s)
64
65         # Sistema linear:
66         # c2*dr + c3*ds = -b1
67         # c1*dr + c2*ds = -b0
68         det = c[2] * c[2] - c[1] * c[3]
69
70         if abs(det) < 1e-14:
71             # Pequena perturbacao para escapar de singularidade numerica

```

```

72         r += 1e-3
73         s -= 1e-3
74         continue
75
76         delta_r = ((-b[1]) * c[2] - c[3] * (-b[0])) / det
77         delta_s = (c[2] * (-b[0]) - (-b[1]) * c[1]) / det
78
79         r += delta_r
80         s += delta_s
81
82         residuo = max(abs(b[0]), abs(b[1]))
83         passo = max(abs(delta_r), abs(delta_s))
84         historico.append((it, r, s, delta_r, delta_s, b[0], b[1],
residuo))
85
86         if passo < tol and residuo < 1e-8:
87             return r, s, b, historico, True
88
89         b = divisao_sintetica_quadratica(a, r, s)
90         return r, s, b, historico, False
91
92
93 def raizes_do_divisor(r, s):
94     """Retorna as duas raizes de  $x^2 - rx - s = 0$ ."""
95     discriminante = complex(r * r + 4.0 * s)
96     raiz_discriminante = cmath.sqrt(discriminante)
97     return (r + raiz_discriminante) / 2.0, (r - raiz_discriminante) /
2.0
98
99
100 def resolver_polinomio_bairstow(a, r0=0.0, s0=-1.0, tol=1e-10, max_iter
=100):
101     """Resolve um polinomio real por deflacao sucessiva via Bairstow."""
102     polinomio = np.array(a, dtype=float)
103     raizes = []
104     historicos = []
105
106     # Tentativas alternativas aumentam a robustez caso o chute inicial
seja ruim.
107     tentativas = [
108         (r0, s0), (0.0, 0.0), (-1.0, -1.0), (1.0, 1.0),
109         (-2.0, -4.0), (2.0, -1.0), (-3.0, -3.0), (3.0, 3.0)
110     ]
111
112     while len(polinomio) - 1 > 2:
113         convergiu = False
114
115         for chute_r, chute_s in tentativas:
116             r, s, b, hist, ok = bairstow_par(polinomio, chute_r, chute_s
, tol, max_iter)
117             if ok:
118                 convergiu = True
119                 break
120
121         if not convergiu:
122             raise RuntimeError("O metodo de Bairstow nao convergiu para
o polinomio atual.")
123

```

```

124     x1, x2 = raizes_do_divisor(r, s)
125     raizes.extend([x1, x2])
126     historicos.append(hist)
127
128     # Deflacao: o quociente e formado pelos coeficientes b2, b3,
..., bn.
129     polinomio = b[2:].copy()
130     polinomio[np.abs(polinomio) < 1e-12] = 0.0
131
132     grau_final = len(polinomio) - 1
133
134     if grau_final == 2:
135         a0, a1, a2 = polinomio
136         discriminante = complex(a1 * a1 - 4.0 * a2 * a0)
137         raizes.append((-a1 + cmath.sqrt(discriminante)) / (2.0 * a2))
138         raizes.append((-a1 - cmath.sqrt(discriminante)) / (2.0 * a2))
139     elif grau_final == 1:
140         a0, a1 = polinomio
141         raizes.append(-a0 / a1)
142
143     return raizes, historicos
144
145
146 def imprimir_raizes(nome, raizes):
147     print(f"\n{n{nome}}")
148     for i, raiz in enumerate(raizes, start=1):
149         print(f"x_{i} = {raiz.real:.10f} {raiz.imag:+.10f}j")
150
151
152 def experimento_convergencia(a):
153     """Testa diferentes chutes iniciais para o polinomio da APC2."""
154     chutes = [(0, -1), (-1, -1), (-2, -4), (1, 1), (-2, 1), (2, -1), (3,
155     3), (-3, -3)]
156     resultados = []
157
158     plt.figure()
159     for r0, s0 in chutes:
160         r, s, b, hist, ok = bairstow_par(np.array(a, dtype=float), r0,
161         s0)
162         iteracoes = [linha[0] for linha in hist]
163         residuos = [max(linha[7], 1e-16) for linha in hist]
164         if iteracoes:
165             plt.semilogy(iteracoes, residuos, marker="o", label=f"r0={r0
166             }, s0={s0}")
167             resultados.append((r0, s0, ok, len(hist), r, s, b[0], b[1]))
168
169     plt.xlabel("Iteracao")
170     plt.ylabel("Maximo residuo |b0|, |b1|")
171     plt.title("Convergencia do Bairstow para o polinomio da APC2")
172     plt.grid(True)
173     plt.legend(fontsize=8)
174     plt.tight_layout()
175     plt.savefig("convergencia_bairstow_ppc2.png", dpi=200)
176
177     return resultados
178
179
180 def fractal_bairstow(a, limite=5.0, pontos=160, tol=1e-8, max_iter=60):

```

```

178     """Gera o mapa de convergencia no plano (r0, s0)."""
179     r_valores = np.linspace(-limite, limite, pontos)
180     s_valores = np.linspace(-limite, limite, pontos)
181     mapa = np.zeros((pontos, pontos), dtype=float)
182
183     for i, s0 in enumerate(s_valores):
184         for j, r0 in enumerate(r_valores):
185             _, _, _, hist, ok = bairstow_par(np.array(a, dtype=float),
186             r0, s0, tol=tol, max_iter=max_iter)
187             mapa[i, j] = len(hist) if ok else np.nan
188
189     plt.figure()
190     plt.imshow(
191         mapa,
192         extent=[-limite, limite, -limite, limite],
193         origin="lower",
194         aspect="auto"
195     )
196     plt.xlabel("r0")
197     plt.ylabel("s0")
198     plt.title("Fractal de Bairstow - numero de iteracoes ate convergir")
199     plt.colorbar(label="Iteracoes")
200     plt.tight_layout()
201     plt.savefig("fractal_bairstow_ppc2.png", dpi=200)
202
203     return mapa
204
205 def main():
206     # Validacao: polinomio de 7o grau com raizes conhecidas:
207     # [-3, -1, 2, 4, 5, 1+2j, 1-2j]
208     # p(x) = x^7 - 9x^6 + 16x^5 + 50x^4 - 219x^3 + 367x^2 + 10x - 600
209     pol_validacao = [-600, 10, 367, -219, 50, 16, -9, 1]
210     raizes_validacao, _ = resolver_polinomio_bairstow(pol_validacao, r0
211     =0.5, s0=-0.5)
212     imprimir_raizes("Raizes obtidas no polinomio de validacao",
213     raizes_validacao)
214
215     # Polinomio caracteristico obtido na APC2:
216     # P(lambda) = 2*lambda^4 + 5*lambda^3 + 12*lambda^2 + 8*lambda + 8
217     pol_apc2 = [8, 8, 12, 5, 2]
218     raizes_apc2, _ = resolver_polinomio_bairstow(pol_apc2, r0=0.0, s0
219     =-1.0)
220     imprimir_raizes("Autovalores obtidos para o polinomio da APC2",
221     raizes_apc2)
222
223     print("\nTeste de convergencia para diferentes chutes iniciais")
224     resultados = experimento_convergencia(pol_apc2)
225     print("r0\t s0\t convergiu\t iter\t r\t s\t b0\t b1")
226     for linha in resultados:
227         print("{:.1f}\t {:.1f}\t {} \t {} \t {:.8f}\t {:.8f}\t {:.2e}\t {:.2e}"
228         .format(*linha))
229
230     print("\nGerando fractal de Bairstow...")
231     fractal_bairstow(pol_apc2)
232     print("Figuras salvas: convergencia_bairstow_ppc2.png e
233     fractal_bairstow_ppc2.png")
234

```

```

229
230 if __name__ == "__main__":
231     main()

```

Listing 1: Implementação do método de Bairstow e geração dos resultados do PPC2.

10 Conclusão

O método de Bairstow foi implementado com sucesso para polinômios reais de grau arbitrário, permitindo a obtenção de raízes reais e complexas conjugadas por meio de divisões sintéticas quadráticas sucessivas. A validação com um polinômio de sétima ordem demonstrou a capacidade do algoritmo de recuperar raízes conhecidas com erro numérico muito pequeno.

Na aplicação ao polinômio característico da APC2, foram obtidos dois pares complexos conjugados de autovalores. Como todos possuem parte real negativa, conclui-se que o sistema massa–mola–amortecedor analisado é estável e apresenta resposta oscilatória amortecida. A análise de convergência e o fractal de Bairstow mostraram que a escolha dos parâmetros iniciais r_0 e s_0 influencia diretamente a velocidade de convergência do método.

Referências

- [1] CHAPRA, S. C.; CANALE, R. P. **Métodos Numéricos para Engenharia**. McGraw-Hill, 5^a edição, 2008.
- [2] GONTIJO, R. G. **Notas de aula do curso de Cálculo Numérico Aplicado**. Universidade de Brasília (UnB), 2026.